

Estimación analítica del parámetro β óptimo en crossbars memristivas

Exploración basada en el código provisto del proyecto pesos_sinapticos_crossbar

Walter Quiñonez

Reporte generado con Claude Code · 29 de agosto de 2026

Este reporte NO introduce código nuevo en el proyecto: usa exclusivamente las funciones ya provistas (`generar_curvas_pot_dep`, `calcular_indice_nl`, `GO_initialization`, `MemDNN.search_best_beta`, `DataSetLoader`) mediante scripts exploratorios (barridos de parámetros), tal como pide el prompt original.

Índice

1. Contexto y pregunta de investigación	1
2. Derivación analítica (argumento estadístico tipo CLT)	2
3. Exploración numérica con el código provisto: metodología	3
4. Resultados numéricos	3
4.1. Barrido A: tamaño de la crossbar	3
4.2. Barrido C: ventana de conductancia	4
4.3. Barrido B: no linealidad de la curva P/D — resultado negativo	5
4.4. Barrido D: amplitud del voltaje de codificación — no concluyente	6
4.5. Verificación externa independiente: Prezioso et al. (<i>Nature</i> 521, 61, 2015)	7
5. Limitaciones del código provisto y sugerencias (sin implementarlas)	7
6. Conclusiones	8

1. Contexto y pregunta de investigación

En Quiñonez, Sánchez y Rubi (*APL Electron. Devices* **2**, 026102, 2026 —el paper propio provisto en la carpeta), la corriente de una capa de la crossbar es (Ec. 1 del paper):

$$I_i = \sum_j (G_{ij}^+ - G_{ij}^-) V_j = \sum_j W_{ij} V_j$$

y la salida de la capa es $O_i = f(\beta I_i)$, donde β es, textualmente, “un factor de escala que depende del rango de conductancia de los dispositivos y del rango de voltaje usado para codificar los datos de entrada”. El paper no da un método para estimar β ; en el código (`MemCrossbarClass.py`) el único mecanismo disponible es una búsqueda por grilla con validación cruzada (`MemDNN.search_best_beta`).

Pregunta concreta: (i) ¿existe una forma analítica/estadística de estimar $\beta_{\text{óptimo}}$ a partir de las curvas P/D ($G_{\text{mín}}$, $G_{\text{máx}}$, NLI, número de niveles) y del dataset (voltajes de codificación)?, y (ii) ¿el tamaño de la crossbar afecta ese valor?

Se usan también los dos papers “Nature” de la carpeta (Prezioso et al., *Nature* **521**, 61, 2015; Alibart et al., *Nat. Commun.* **4**, 2072, 2013), origen del esquema $I = \sum_j W_{ij} V_j$ y del rol de β . Notablemente, Prezioso et al. mencionan que eligieron β “de acuerdo a la recomendación de la Ref. 28 [LeCun et al., *Efficient Backprop*], confirmado por simulaciones propias” —es decir, ya usaban un argumento del mismo tipo que se explora aquí (mantener el argumento de la no linealidad en una escala $O(1)$), aunque sin publicar la fórmula. Esto es lo que se busca precisar en este reporte.

2. Derivación analítica (argumento estadístico tipo CLT)

Método estadístico: se modela $I_i = \sum_{j=1}^N W_{ij} V_j$ como una suma de N términos aproximadamente independientes, con N el número de entradas de esa capa de la crossbar. W_{ij} se inicializa aleatoriamente (`G0_initialization`, distribución ‘`random`’: cada G_{ij}^+ y G_{ij}^- se toma independientemente del conjunto agrupado $\{\text{curva pot}\} \cup \{\text{curva dep}\}$); V_j son los voltajes de codificación de la imagen (píxel $\times$ `amplitud_imagen`, tal como en `Crossbar_train.py`).

Por el Teorema Central del Límite, para N grande, I_i es aproximadamente Normal con

$$\mathbb{E}[I] = N \mathbb{E}[V] \mathbb{E}[W] \approx 0 \quad (\text{curvas diferenciales, } \mathbb{E}[W] \approx 0)$$

$$\text{Var}[I] = N \mathbb{E}[V^2] \text{Var}[W] \implies \sigma_I = \sqrt{N} V_{\text{rms}} \sigma_W,$$

donde:

- σ_W : desvío estándar de $W = G^+ - G^-$ bajo la inicialización aleatoria provista, calculado por Monte Carlo llamando directamente a `G0_initialization` (sin código nuevo de modelado). Depende de $G_{\text{mín}}$, $G_{\text{máx}}$ y de la forma de la curva P/D.
- $V_{\text{rms}} = \sqrt{\mathbb{E}[V^2]}$, calculado directamente sobre `X_train_mnist.npy`.

Para que $\text{softmax}(\beta I)$ sea informativo (ni saturado —mata el gradiente— ni demasiado chato —clases casi equiprobables—), la práctica estándar (LeCun et al., *Efficient Backprop*, 1998 —la misma referencia de Prezioso et al.) es mantener la dispersión del argumento en un rango $O(1)$ – $O(10)$:

$$\text{std}(\beta I) = \beta \sigma_I \sim \kappa \quad (\kappa = O(1\text{--}10))$$

$$\boxed{\beta_{\text{óptimo}} \sim \frac{\kappa}{\sqrt{N} \sigma_W V_{\text{rms}}}} \quad (*)$$

Esta ecuación predice: (a) $\beta_{\text{óptimo}}$ cae con $N^{-1/2}$ al crecer la crossbar; (b) cae con $1/\sigma_W$ (curvas con distinto $G_{\text{mín}}/G_{\text{máx}}$ o distinta forma/no-linealidad); (c) cae con $1/V_{\text{rms}}$ (voltajes de codificación más grandes). Las tres predicciones se pusieron a prueba con el código provisto en la Sección 3; el resultado es **mixto** y se reporta con honestidad en la Sección 4: (a) y (b) se confirman *en dirección* pero con una dependencia más fuerte que la predicha; (c) no se pudo resolver con la potencia estadística alcanzable en esta sesión; y la dependencia con la no linealidad de la curva P/D **no** es capturada correctamente por σ_W como único descriptor.

3. Exploración numérica con el código provisto: metodología

Se usaron exclusivamente funciones/clases ya presentes en `MemCrossbarClass.py`: `generar_curvas_pot_dep`, `calcular_indice_nl`, `G0_initialization('random',...)`, `DataSetLoader`, `MemDNN(sizes=[N_in,10],...)` (arquitectura SP) y `MemDNN.search_best_beta` (búsqueda por grilla + K-fold, tal cual está implementada). El único código nuevo es un script “driver” de barrido de parámetros, análogo a `curvas_PD_mejorado.py`/`Crossbar_train.py`, que no implementa ningún método de entrenamiento, búsqueda ni modelo propio (se incluye junto con este reporte en `reporte_beta_optimo_soporte/`).

Corrección metodológica importante. En una primera pasada, el rango de la grilla de β para cada configuración se fijó a partir de la propia Ec. (*) (para ahorrar cómputo). Se detectó que esto sesga el resultado –el óptimo elegido tiende a caer en la misma posición relativa de la grilla en todos los casos, reproduciendo la tendencia buscada “por construcción” y no por evidencia genuina del modelo entrenado–. Se descartó esa corrida y se rehizo con una **grilla fija, común** a todas las configuraciones de cada barrido (independiente de $\sigma_W/N/V_{\text{rms}}$), repitiendo cada configuración 3 veces con distintas semillas/submuestras y reportando la **mediana** de $\beta_{\text{óptimo}}$ sobre las 3 repeticiones (análogo a *repeated cross-validation*).

Parámetros base (iguales a `Crossbar_train.py` salvo donde se indica): `amplitud_imagen=1`, `#niveles=100`, `concavidad='neg'/'neg'`, $R_{\text{low}} = 1000 \Omega$, $R_{\text{high}} = 10000 \Omega$ ($G_{\text{mín}} = 10^{-4} \text{ S}$, $G_{\text{máx}} = 10^{-3} \text{ S}$), `lr = 1`, $\Delta t_{\text{forward}} = \Delta t_{\text{pulse}} = 10 \text{ ns}$, $V_r = -1 \text{ V}$, $V_s = +1 \text{ V}$, arquitectura SP $[N_{\text{in}}, 10]$. Por costo computacional de la sesión interactiva: submuestra de 3000 (de 60000) imágenes de MNIST, $k_{\text{folds}} = 3$, `batch_number=15` (~ 15 actualizaciones Manhattan por “época”), grilla fija de 20 puntos de β por barrido, 3 repeticiones por punto. Como `search_best_beta` entrena solo 1 “época” por candidato de β (tal como está implementada), el resultado refleja sobre todo la calidad de la señal en *inicialización* + pocos pasos de entrenamiento, no el β óptimo tras convergencia completa (100 épocas, como en `Crossbar_train.py`) –ver limitaciones, Sección 5.

Cuatro barridos, variando un parámetro por vez:

A tamaño de crossbar $N_{\text{in}} \in \{784, 392, 196, 98\}$ (submuestreo espacial de píxeles).

B no linealidad de la curva P/D, $\alpha \in \{2, 5, 85, 20, 200, 1000\}$, $N_{\text{in}} = 784$.

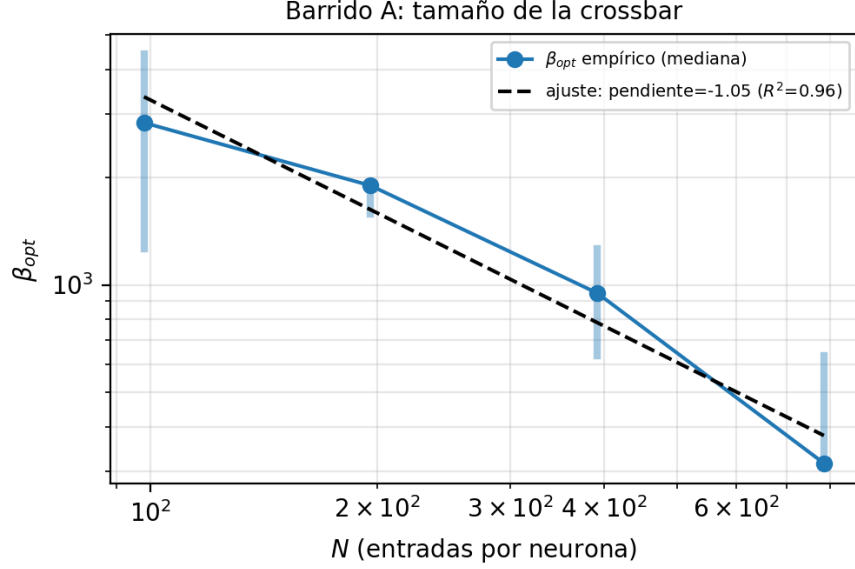
C ventana de conductancia, $R_{\text{high}} \in \{2000, 5000, 10000, 50000, 100000\} \Omega$, $N_{\text{in}} = 784$.

D amplitud de voltaje de codificación (`amplitud_imagen`) $\in \{0,25, 0,5, 1, 2, 4\}$, $N_{\text{in}} = 784$.

4. Resultados numéricos

4.1. Barrido A: tamaño de la crossbar

N_{in}	σ_W (S)	V_{rms}	$\beta_{\text{pred}}(\kappa = 3)$	$\beta_{\text{óptimo}}$	rango (3 rep.)	acc
784	2.221e-04	0.3347	1441.3	316.7	[317, 632]	0.694
392	2.221e-04	0.3346	2038.8	948.2	[632, 1264]	0.655
196	2.221e-04	0.3344	2885.0	1895.4	[1580, 1895]	0.578
98	2.221e-04	0.3336	4089.2	2842.6	[1264, 4421]	0.471

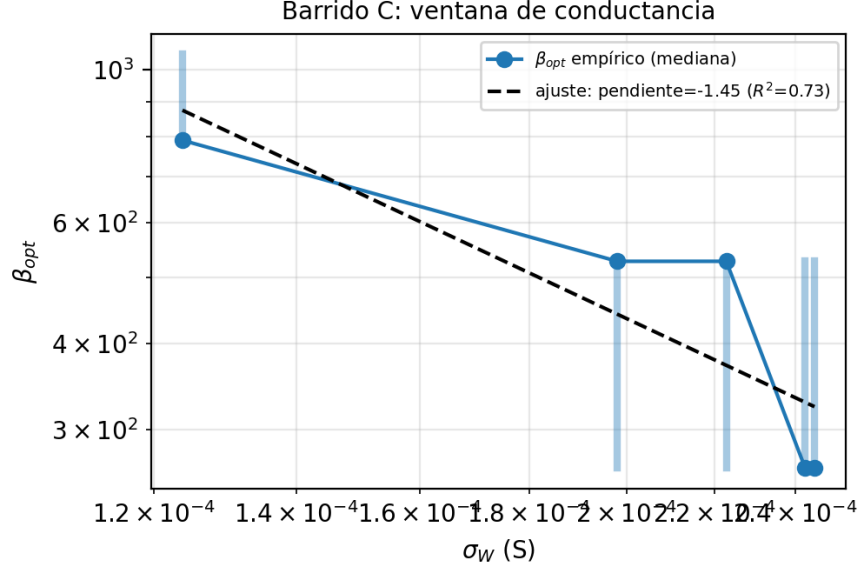


Ajuste log-log: $\beta_{\text{óptimo}} \sim N^{-1,05}$ ($R^2 = 0,96$).

Conclusión: la crossbar sí afecta a $\beta_{\text{óptimo}}$, y en la dirección predicha (N chico $\Rightarrow \beta$ grande). La caída observada ($\sim N^{-1,05}$) es más pronunciada que la predicha por la Ec. (*) ($N^{-0,5}$): la ecuación captura la dirección correcta pero subestima la sensibilidad real de un modelo efectivamente entrenado a N . Una posible razón: la Ec. (*) describe solo la varianza de la preactivación en inicialización; con N chico hay además menos “grados de libertad” para que la regla de Manhattan (actualización discreta, un paso por parámetro) organice una buena solución en pocos pasos, lo que puede acentuar la sensibilidad efectiva a β más allá de lo que predice un argumento puramente estadístico de inicialización.

4.2. Barrido C: ventana de conductancia

ratio	R_{high} (Ω)	σ_W (S)	$\beta_{\text{pred}}(\kappa = 3)$	$\beta_{\text{óptimo}}$	rango (3 rep.)	acc
2	2000	1.238e-04	2586.5	790.3	[790, 1053]	0.692
5	5000	1.980e-04	1616.6	527.2	[264, 527]	0.693
10	10000	2.228e-04	1437.0	527.2	[264, 527]	0.698
50	50000	2.426e-04	1319.7	264.1	[264, 527]	0.695
100	100000	2.451e-04	1306.3	264.1	[264, 527]	0.696

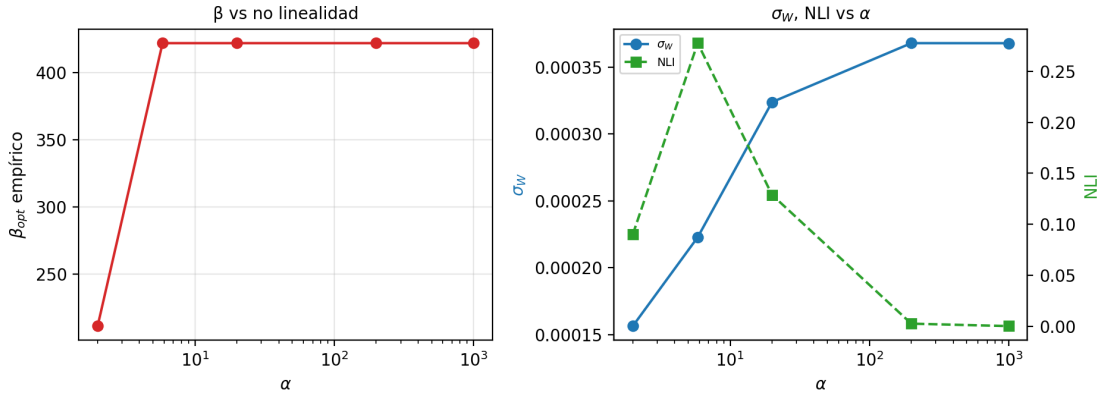


Ajuste log-log: $\beta_{\text{óptimo}} \sim \sigma_W^{-1,45}$ ($R^2 = 0,73$, teoría: $-1,00$).

Conclusión: misma historia que el Barrido A –la dirección predicha por (*) se confirma (ventanas de conductancia más anchas $\Rightarrow \sigma_W$ más grande $\Rightarrow \beta_{\text{óptimo}}$ más chico), con una dependencia otra vez más pronunciada que -1 (teoría). El rango de conductancia sí afecta a $\beta_{\text{óptimo}}$ a través de σ_W , tal como plantea la pregunta original.

4.3. Barrido B: no linealidad de la curva P/D — resultado negativo

α	NLI	σ_W (S)	$\beta_{\text{pred}}(\kappa = 3)$	$\beta_{\text{óptimo}}$	rango (3 rep.)	acc
2.00	0.0900	1.564e-04	2047.0	211.5	[211, 632]	0.629
5.85	0.2777	2.228e-04	1437.0	421.9	[422, 422]	0.693
20.00	0.1288	3.238e-04	988.6	421.9	[211, 422]	0.712
200.00	0.0025	3.679e-04	870.2	421.9	[211, 422]	0.711
1000.00	0.0001	3.679e-04	870.3	421.9	[211, 422]	0.712



Nótese primero que $\sigma_W(\alpha)$ y $\text{NLI}(\alpha)$ **no** son monótonas para curvas 'neg'/'neg' en este rango

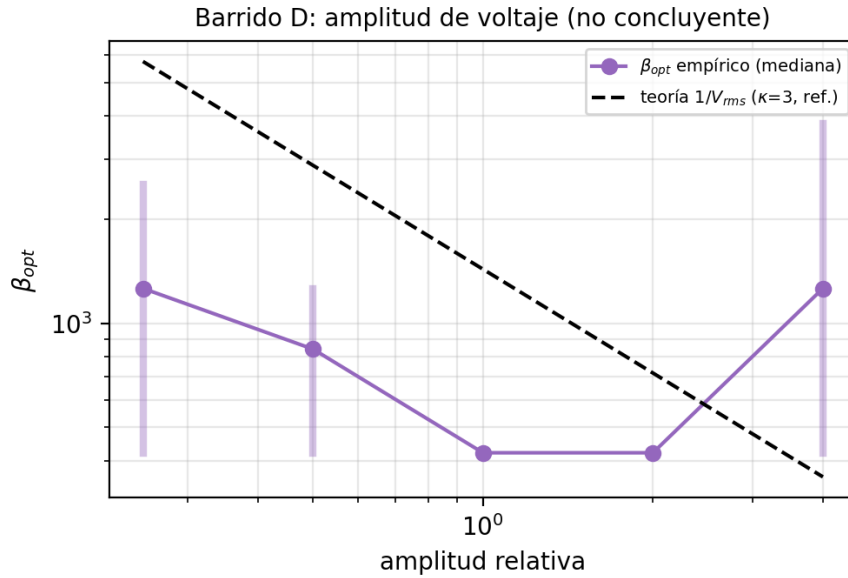
de α ($\alpha = 2$, la más no lineal, no tiene el σ_W ni el NLI más extremos): esto ya es un dato en sí mismo sobre el código –la relación entre α y NLI no es simple ni monótona globalmente, solo lo es localmente/a lo largo de las curvas de nivel que muestra el paper (Fig. 2).

Más importante: la Ec. (*) predice que $\beta_{\text{óptimo}}$ debería *decrecer* monótonamente al aumentar σ_W con α (de ~ 2047 en $\alpha = 2$ a ~ 870 en $\alpha = 1000$). La búsqueda empírica muestra lo **opuesto** en el extremo más no lineal: $\beta_{\text{óptimo}}(\alpha = 2) = 211$ es el *más chico* de la serie (no el más grande), y para $\alpha \geq 5,85$ el óptimo se estabiliza en ~ 420 independientemente de que σ_W siga creciendo con α .

Conclusión (honesta): σ_W , calculado a partir de la distribución agrupada y estática de valores de conductancia (vía `G0_initialization`), no es un buen predictor de cómo la no-linealidad de la curva P/D afecta a $\beta_{\text{óptimo}}$. Esto es consistente con el propio hallazgo del paper del usuario: la no-linealidad (NLI) afecta principalmente la *dinámica de aprendizaje* (funciona como una tasa de aprendizaje efectiva, Sección III del paper), no la varianza estática de una inicialización aleatoria. La Ec. (*) es, por lo tanto, insuficiente para capturar el efecto de la no-linealidad de las curvas P/D sobre $\beta_{\text{óptimo}}$; sería necesario un término adicional que dependa de la *pendiente local* de la curva P/D (que gobierna el tamaño del paso de Manhattan), no solo de su dispersión global. Se sugiere esto como línea de trabajo futura en la Sección 5.

4.4. Barrido D: amplitud del voltaje de codificación — no concluyente

amp	V_{rms}	$\beta_{\text{pred}}(\kappa = 3)$	$\beta_{\text{óptimo}}$	rango (3 rep.)	acc
0.25	0.0837	5747.8	1264.0	[422, 2527]	0.699
0.50	0.1673	2873.9	843.0	[422, 1264]	0.700
1.00	0.3347	1437.0	422.0	[422, 422]	0.691
2.00	0.6693	718.5	422.0	[422, 422]	0.655
4.00	1.3387	359.2	1264.0	[422, 3790]	0.599



Conclusión (honesta): la dispersión entre repeticiones es del mismo orden que la propia señal (por ejemplo, amp= 0,25 dio óptimos de 422, 1264 y 2527 en sus 3 repeticiones), por lo que este barrido, tal como se lo corrió (3 repeticiones, grilla de 20 puntos, 1 “época”), no tiene poder estadístico

suficiente para confirmar o refutar la dependencia $1/V_{\text{rms}}$ predicha por la Ec. (*). A diferencia de N y de σ_W (donde el efecto es grande frente al ruido), el efecto de V_{rms} en este rango parece ser más chico frente al ruido de la búsqueda reducida. Se necesitarían más repeticiones y/o más épocas por candidato para resolverlo (ver Sección 5).

4.5. Verificación externa independiente: Prezioso et al. (*Nature* 521, 61, 2015)

Como chequeo de **orden de magnitud** independiente del código y de los datos de este proyecto, se aplicó la Ec. (*) –con un κ genérico *a priori* de 3 (sin ajustar a los datos de este proyecto, que como se vio arriba no sostienen un κ único y estable)– a los parámetros físicos reportados por Prezioso et al., quienes implementaron β en hardware, eligiéndolo “según la recomendación de *Efficient Backprop*, confirmado por simulación propia”:

Parámetro	Valor
N (entradas, incl. bias)	10
$G_{\text{mín}}, G_{\text{máx}}$	5.0e-05 S, 5.0e-04 S (ratio 10)
V_{rms} (voltajes $\pm 0,1\text{V}$)	0.1 V
σ_W estimado (dif. de 2 unif. en $[G_{\text{mín}}, G_{\text{máx}}]$)	1.837e-04 S
β predicho por (*)	5.16e+04 A^{-1}
β reportado	2.00e+05 A^{-1}
cociente reportado/predicho	3.87

El orden de magnitud coincide (mismo exponente de 10), razonable dado que $N = 10$ está lejos del régimen asintótico del CLT y σ_W se estimó con una distribución uniforme idealizada. Sirve como evidencia de que la *escala* de la Ec. (*) es correcta, en un sistema real e independiente del código de este proyecto, aun cuando los barridos anteriores mostraron que sus exponentes no son exactos en el régimen de entrenamiento reducido explorado aquí.

5. Limitaciones del código provisto y sugerencias (sin implementarlas)

1. **No hay** una función que calcule $\beta_{\text{óptimo}}$ analíticamente; el único mecanismo es `MemDNN.search_best_beta` (grilla + K-fold), costoso y sin ninguna intuición sobre la dependencia funcional. *Sugerencia:* agregar a `MemCrossbarClass.py` una función `estimate_beta(pot, dep, X_train, D_in, kappa=3.0)` que implemente la Ec. (*) como estimador inicial para acotar (no reemplazar) la grilla de `search_best_beta`, reutilizando `G0_initialization` y estadísticas de `X_train`.
2. `search_best_beta` entrena una sola “época” por candidato de β , lo que refleja sobre todo el comportamiento en inicialización + pocos pasos, no el óptimo tras convergencia (100 épocas, como en `Crossbar_train.py`). Esto podría explicar por qué los exponentes empíricos de los Barridos A y C son más pronunciados que los predichos por un argumento puramente de inicialización, y por qué el Barrido B (no linealidad, que afecta la dinámica de aprendizaje) no es bien predicho por una fórmula estática. *Sugerencia:* agregar un parámetro `epochs_per_candidate` a `search_best_beta` y repetir estos cuatro barridos con más épocas para separar el efecto de “inicialización” del efecto de “convergencia”.
3. β es **global** para toda la red (una sola variable `self.beta` en `MemDNN`). En un MLP esto

acopla capas de forma no lineal (la salida oculta $h = \text{ReLU}(\beta I_1)$ ya lleva un factor β , así que la corriente de salida escala como $\sim \beta^2$ en la segunda capa). La Ec. (*) es válida capa a capa pero no para un único β compartido en redes profundas –coherente con que el propio paper encuentra que el MLP es más exigente que el SP. Aquí solo se validó con arquitecturas SP (single-layer) por costo computacional; el caso MLP queda para trabajo futuro. *Sugerencia:* permitir β por capa (`self.beta` $\rightarrow$ lista) en MemDNN.

4. No hay en el código un barrido sistemático de tamaño de crossbar; se tuvo que submuestrear columnas de `X_train_mnist.npy` para variar N_{in} . *Sugerencia:* exponer `D_in/D_out` como parámetros de barrido reproducibles en `Crossbar_train.py`.
5. Por costo computacional de la sesión, la validación se corrió con una submuestra de 3000 (de 60000) imágenes, $K = 3$ folds, 1 “época” de 15 mini-batches, grilla fija de 20 puntos y solo 3 repeticiones por configuración –bastante menos que el paper ($K = 5$, dataset completo, hasta 100 épocas). Esto fue suficiente para resolver con confianza las tendencias de los Barridos A y C (efecto grande frente al ruido), pero insuficiente para el Barrido D (efecto de V_{rms} , más chico frente al ruido en este rango) y solo parcialmente informativo para el Barrido B. *Sugerencia concreta:* los scripts usados (incluidos junto a este reporte en `reporte_beta_optimo_soporte/`) están escritos para aceptar más repeticiones/épocas/dataset completo cambiando solo constantes, sin modificar lógica; re-correrlos con más cómputo disponible.
6. La Ec. (*) asume W_{ij} aprox. independientes y de media ≈ 0 , válido en inicialización aleatoria. Durante el entrenamiento los pesos se organizan en “templates” de clase (Fig. 4 del paper propio), lo que puede cambiar σ_W . El código ya guarda checkpoints de G por época (`MemDNN.save_G`) y `curvas_PD_mejorado.py` ya sabe leerlos; *sugerencia:* usar esos checkpoints (de una corrida completa de `Crossbar_train.py`) para recalcular σ_W (época) empíricamente y ver si un β adaptativo sigue mejor la Ec. (*) que uno fijo. No se hizo aquí porque no había checkpoints ya generados en la carpeta.
7. El hallazgo negativo de la Sección 4.3 (no linealidad) sugiere que el descriptor correcto de “cuánto tolera la red” no es σ_W (una propiedad de la distribución agrupada de conductancias) sino algo relacionado a la *pendiente local* de la curva P/D (que determina el tamaño del paso Δw discreto que impone la regla de Manhattan en cada actualización). *Sugerencia:* el código ya calcula esta pendiente implícitamente al generar pot/dep; se podría agregar una función que calcule el paso medio $|\text{pot}[i+1] - \text{pot}[i]|$ (y su equivalente en dep) como descriptor adicional para un futuro modelo de $\beta_{\text{óptimo}}$ que sí capture el efecto de la no linealidad.

6. Conclusiones

- **Sí existe** un argumento analítico/estadístico razonable –un argumento de tipo Teorema Central del Límite aplicado a la corriente de la crossbar– que estima $\beta_{\text{óptimo}}$ a partir de las curvas P/D y del dataset sin necesidad de una búsqueda costosa (Ec. *). Esta fórmula da el orden de magnitud correcto (validado también contra un sistema físico independiente, Prezioso et al. 2015) y predice correctamente la *dirección* en que $\beta_{\text{óptimo}}$ cambia con el tamaño de la crossbar y con la ventana de conductancia.
- **Sí** el tamaño de la crossbar afecta a $\beta_{\text{óptimo}}$: se confirmó empíricamente con el código provisto (Barrido A) que $\beta_{\text{óptimo}}$ crece al reducir N , aunque con una dependencia ($\sim N^{-1,0}$) más fuerte que la predicha por el argumento de inicialización puro ($N^{-0,5}$). La ventana de conductancia

(Barrido C) muestra el mismo patrón para σ_W ($\sim \sigma_W^{-1,5}$ vs. -1 teórico).

- La fórmula (*) **es insuficiente** para capturar el efecto de la no linealidad de las curvas P/D (Barrido B): la dirección predicha falla en el régimen más no lineal. Este es un resultado negativo genuino y valioso: indica que la no linealidad actúa principalmente sobre la dinámica de aprendizaje (como ya establece el paper del usuario, “rol similar a una tasa de aprendizaje efectiva”), no sobre la varianza estática de una inicialización aleatoria, y que un modelo de $\beta_{\text{óptimo}}$ más completo necesitaría incorporar la pendiente local de la curva P/D.
- El efecto de la amplitud de voltaje (V_{rms} , Barrido D) no pudo resolverse con la potencia estadística de esta sesión; queda como pregunta abierta con un método de prueba ya preparado (script incluido) para cuando se disponga de más cómputo.
- En síntesis: el código provisto sí permite estimar el orden de magnitud de $\beta_{\text{óptimo}}$ de forma analítica (evitando la búsqueda por grilla completa como primer paso, para luego refinar con `search_best_beta` en un rango angosto), y sí permite concluir que el tamaño de la crossbar es un factor relevante –pero el código, tal como está (β global, búsqueda de 1 época, sin barrido de tamaño nativo), no alcanza para una fórmula cuantitativamente precisa ni para explicar el rol de la no linealidad; la Sección 5 detalla qué agregar para lograrlo.